Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

AI-Powered Premier League Match Prediction System

By
Mohammad Elmahi

April 26, 2026

B.Sc. (Hons) in Software Development

Minor Dissertation

Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.

Contents

1 Introduction **2**
1.1 Context and Background . . . 2
1.2 Project Objectives . . . 3
1.3 Success Metrics . . . 4
1.4 What This Project Built . . . 5
1.5 Research Questions . . . 6
1.6 Real-World Performance . . . 6
1.7 Key Challenges . . . 7
1.8 Dissertation Structure . . . 7

2 Methodology **9**
2.1 Development Approach . . . 9
2.2 Requirements Gathering . . . 9
2.3 Data Collection Strategy . . . 10
2.3.1 Historical Match Data . . . 10
2.3.2 Live League Data . . . 10
2.4 Machine Learning Methodology . . . 11
2.4.1 Train-Test Split . . . 11
2.4.2 Model Selection . . . 11
2.4.3 Feature Engineering . . . 11
2.5 Technology Selection . . . 12
2.6 Testing and Validation . . . 13
2.7 Problem-Solving Approach . . . 13
2.8 Summary . . . 14

3 Technology Review **15**
3.1 Football Prediction Research . . . 15
3.2 Machine Learning for Classification . . . 16
3.2.1 Logistic Regression . . . 17
3.2.2 Random Forest . . . 17
3.2.3 Decision Tree . . . 18
3.2.4 Class Imbalance . . . 19
3.3 Feature Engineering in Sports Prediction . . . 19
3.3.1 Form Metrics . . . 19
3.3.2 Positional Features . . . 20
3.3.3 Draw Rate Features . . . 21
3.3.4 Home Advantage . . . 21
3.4 Backend Technologies . . . 22
3.4.1 Python and scikit-learn . . . 22
3.4.2 Flask and RESTful APIs . . . 22
3.4.3 PostgreSQL . . . 23

3.6 Deployment and Cloud Services 24
3.7 Summary 25

4 System Design **27**
4.1 System Architecture Overview 27
4.2 Backend API Design 29
4.2.1 API Endpoint Specifications 29
4.2.2 Error Handling 31
4.3 Database Schema 31
4.4 Feature Engineering Workflow 33
4.5 Frontend Component Structure 34
4.5.1 Dashboard Component 34
4.5.2 History Component 34
4.5.3 Model Comparison Component 34
4.5.4 Team Stats Component 34
4.6 User Interface Screenshots 35
4.7 Deployment Configuration 36
4.8 Summary 37

5 System Evaluation **38**
5.1 Prediction Accuracy Results 38
5.2 Comparison to Benchmarks 39
5.3 Feature Importance Analysis 40
5.4 Model Comparison 41
5.5 Case Study: Gameweek 29 Failure 42
5.6 Case Study: Gameweek 31 Success 43
5.7 Assessment Against Objectives 45
5.8 Identified Limitations 46
5.9 Summary 46

6 Conclusion **48**
6.1 Project Summary 48
6.2 Key Findings 49
6.2.1 Prediction Accuracy Exhibits High Variance 49
6.2.2 Positional Features Dominate Importance 50
6.2.3 Home Advantage Overestimation 50
6.2.4 Draw Prediction Remains Challenging 51
6.3 Project Outcomes 51
6.4 Unexpected Discoveries 52
6.5 Future Work 53
6.6 Reflection 54

6.7 Final Remarks 55

A Project Repository and Installation 59
A.1 Source Code Repository 59
A.2 Live Application 59
A.3 Installation Instructions 60
A.3.1 Prerequisites 60
A.3.2 Backend Setup 60
A.3.3 Frontend Setup 60
A.3.4 Generating Predictions 60

List of Figures

4.1 System Architecture Overview showing data flow between components 28
4.2 Predictions Table Schema 32
4.3 Feature Engineering Workflow 33
4.4 Dashboard displaying upcoming predictions and accuracy statistics 35
4.5 History page showing past predictions color-coded by correctness . . 35
4.6 Model comparison page listing algorithms and feature importance . 36
4.7 Team Stats page displaying live Premier League standings 36

List of Tables

5.1 Prediction accuracy by gameweek 38
5.2 Top 10 features by Logistic Regression coefficient magnitude (remaining 6 features had coefficients < 0.15) 40
5.3 Model comparison on historical test data 41
5.4 Gameweek 29 predictions versus actual results 42

Chapter 1

Introduction

1.1 Context and Background

Predicting the outcome of football matches has captivated fans, analysts, and researchers for decades. The English Premier League, as one of the most competitive football leagues globally, generates billions in betting revenue annually—estimates suggest the UK sports betting market alone exceeds £14 billion per year, with football comprising the largest segment.Professional bookmakers invest heavily in sophisticated prediction models, employing teams of statisticians and data scientists. Yet despite this investment and access to proprietary data sources, even the most advanced commercial systems struggle to exceed 55% accuracy [1]—only marginally better than an informed guess. This ceiling presents an intriguing challenge: can machine learning, using publicly available data, approach industry-standard prediction accuracy?

For readers unfamiliar with football prediction, the task involves forecasting one of three mutually exclusive outcomes before a match begins: home win, away win, or draw. Unlike sports such as basketball where scoring is frequent and outcomes tend toward statistical means, football matches often hinge on single goals scored in brief moments. A match can feature 30 shots yet end 0-0, or see a team dominate possession yet lose to a single counter-attack. Each match is influenced by quantifiable factors—team form over recent matches, league position, goals scored and conceded, home advantage—alongside intangible variables including injuries, tactical adjustments, referee decisions, weather conditions, and crowd atmosphere. This combination of measurable patterns and irreducible uncertainty makes football an ideal testbed for machine learning: complex enough to warrant sophisticated models, data-rich enough to make the problem tractable, yet sufficiently unpredictable to expose the limitations of purely data-driven approaches.

The Premier League specifically presents unique challenges for prediction. Un-

like less competitive leagues where a handful of elite teams dominate, the Premier League exhibits high parity across the table. On any given matchday, teams near the bottom can defeat those competing for championships—a phenomenon colloquially known as "any team can beat any team." This competitive balance, while entertaining for fans, creates substantial noise in prediction models. Historical form becomes less reliable when mid-table teams regularly upset title contenders.

This project is relevant for several reasons. First, it demonstrates the practical application of supervised machine learning to real-world, high-variance problems where even marginal improvements have commercial value. In betting markets, a model achieving 52% accuracy versus 50% represents the difference between profit and loss over thousands of predictions. Second, it explores the fundamental limitations of ML when confronted with systems that are genuinely unpredictable—a critical consideration for deploying AI in production environments where overpromising on accuracy can have consequences. Third, from a software engineering perspective, it showcases full-stack development competencies: building a complete web application with real-time API integration, database management, responsive frontend design, and cloud deployment—skills directly transferable to industry roles.

1.2 Project Objectives

This dissertation presents the design, implementation, and evaluation of an AI-powered Premier League match prediction system. The system was built iteratively over the 2025-26 academic year and evaluated against live match results during gameweeks 27-33. The specific objectives established at project outset were:

1. **Build a machine learning model** capable of predicting match outcomes (home win, draw, away win) using historical match data and live league standings, with a target accuracy of 40-50% on real-world predictions

2. **Engineer meaningful features** that capture team performance across multiple dimensions, including short-term form metrics (last 5 matches), positional data (league standings), goal-scoring patterns, and draw tendencies

3. **Integrate external data sources** through the football-data.org API to access live league tables and fixture information, demonstrating RESTful API consumption and data synchronization

4. **Deploy a production-ready web application** that automatically generates predictions for upcoming gameweeks and presents them through an intuitive user interface accessible to non-technical users

5. **Evaluate real-world performance** by comparing predictions against actual match results across multiple gameweeks, tracking accuracy over time, and analyzing which types of predictions succeed or fail

6. **Identify model limitations** through critical analysis of incorrect predictions, exploring whether patterns emerge in failures

1.3 Success Metrics

The project's success is measured using both quantitative and qualitative criteria. Quantitatively, the primary metric is **prediction accuracy**: the percentage of correct outcome predictions across all evaluated matches.Academic literature suggests typical accuracy ranges of 40-50% [2, 3], while professional bookmaker models achieve 50-55%. These benchmarks acknowledge that perfect prediction is impossible—football's inherent randomness ensures even theoretically optimal models would plateau below 60%.

Secondary quantitative metrics include:

- **Model test accuracy**: Performance on historical test data held out from the training set (target: >50%)

- **Feature importance rankings**: Identification of which features contribute most to prediction quality

- **Confidence-based accuracy**: Performance when filtering predictions by model confidence

Qualitative success criteria include:

- **System reliability**: Ability to generate predictions for new gameweeks without manual intervention

- **Deployment accessibility**: Public URL hosting a functioning web application

- **Code quality**: Well-structured, documented codebase with clear separation of concerns

1.4 What This Project Built

The deliverable is a full-stack web application consisting of six integrated components:

Backend API (Python/Flask): The backend exposes RESTful endpoints for retrieving predictions (`/api/predictions`), historical results (`/api/history`), model comparison statistics (`/api/models`), overall accuracy (`/api/accuracy`), and live league standings (`/api/standings`). Flask was chosen over heavier frameworks like Django due to its simplicity—the project requires API endpoints rather than full-featured web views.

Machine Learning Pipeline: The ML pipeline trains three models—Logistic Regression [4], Random Forest [5] (200 trees), and Decision Tree—on 2,261 Premier League matches from seasons 2023-24, 2024-25, and 2025-26. Training uses a temporal split: 628 matches for training, 1,411 for testing. Logistic Regression emerged as the best-performing model with 59.74% training accuracy, beating Random Forest (49.33%) and Decision Tree (37.07%). The trained models are serialized to `.pkl` files and loaded by the prediction service.

Feature Engineering System: The system calculates 16 features per match by analyzing historical data and fetching live league positions. Features include form metrics (points from last 5 matches), goal statistics (average goals scored and conceded), draw rates (percentage of recent matches ending in draws), form-based position estimates, and—critically—actual league table positions fetched in real-time from the football-data.org API. This category provided additional predictive signal, complementing the form-based position proxies already calculated from historical data. **PostgreSQL Database:** Match predictions, actual results, and accuracy statistics are stored in a PostgreSQL database hosted on Render.com's managed database service. The schema centers on a `predictions` table containing home team, away team, date, prediction (home/draw/away), probability distributions, actual result, correctness flag, and gameweek number. This structure enables historical querying, accuracy tracking over time, and comparison between predicted and actual outcomes.

React Frontend: The user interface is built with React 19 and styled using Tailwind CSS. Four main pages provide different views: the Dashboard displays upcoming predictions and overall accuracy; the History page shows past predictions grouped by gameweek with color-coded results (green for correct, red for incorrect); the Models page compares the three algorithms and lists all 16 features ordered by importance; and the Team Stats page presents the live Premier League table with visual highlighting for Champions League (top 4) and relegation (bottom 3) positions.

Deployment Infrastructure: The entire system is deployed on Render.com. The backend runs as a Gunicorn-served Flask application, the frontend builds

to static files served via Render's CDN, and the database operates on Render's managed PostgreSQL service. GitHub integration enables continuous deployment: pushing to the `main` branch triggers automatic rebuilds and deployments.

The system is publicly accessible at `https://fyp-frontend-wgcl.onrender.com` and the complete source code is available at `https://github.com/mohelmahi02/FYP`. The repository contains `/services` (backend API), `/frontend` (React application), `/models` (trained model files), and `/data` (historical match data and fixtures).

1.5 Research Questions

This project investigates four central questions:

1. Can machine learning match industry-standard prediction accuracy using only publicly available data? Professional bookmakers leverage proprietary information—confirmed team lineups, injury reports, and real-time betting market movements. This project deliberately restricts itself to public data to assess whether ML can compete under these constraints.

2. Which features are most predictive of match outcomes? Does recent form matter more than overall league position? Are goal-scoring metrics more predictive than defensive statistics?

3. How does prediction accuracy vary across gameweeks, and what patterns emerge? Tracking performance over multiple gameweeks reveals whether the model maintains consistent accuracy or exhibits volatility.

4. What are the fundamental limitations of data-driven football prediction? Some matches are inherently unpredictable due to red cards, last-minute goals, or referee errors. Understanding where the model fails illuminates the boundary between pattern recognition and genuine uncertainty.

1.6 Real-World Performance

Between February and April 2026, the system generated predictions for gameweeks 27-33 of the Premier League season. Across 68 predictions, the model achieved **36.76% accuracy** (25 correct predictions). Performance varied significantly by gameweek:

- **Gameweek 27**: 30% (3/10 correct)

- **Gameweek 28**: 40% (4/10 correct)
- **Gameweek 29**: 10% (1/10 correct) — Model over-predicted home wins when away teams dominated
- **Gameweek 30**: 30% (3/10 correct)
- **Gameweek 31**: 50% (4/8 correct) — Matching industry-standard accuracy
- **Gameweek 32**: 40% (4/10 correct)
- **Gameweek 33**: 60% (6/10 correct) — Best performance, exceeding industry-standard accuracy

This variability demonstrates both the challenge of football prediction and the value of longitudinal evaluation. A single gameweek provides limited insight given sample sizes of 8-10 matches. The improvement in GW33 to 60% accuracy—exceeding professional benchmarks—suggests the model can perform competitively when match conditions align favorably with historical patterns. Conversely, GW29's collapse to 10% illustrates how football can deviate dramatically from expected patterns.

The 36.76% overall accuracy approaches the 40-50% target established in project objectives. Real-world deployment exposes models to distribution shifts and fundamental randomness that no amount of training data can capture. The gap between 59.74% training accuracy and 36.76% real-world accuracy highlights the challenge of generalization in sports prediction.

1.7 Key Challenges

Several technical challenges emerged during development. Different data sources used inconsistent team naming conventions ("Spurs" vs "Tottenham Hotspur"), requiring a normalization dictionary. The football-data.org API's rate limiting (10 requests/minute) necessitated caching strategies. The model initially never predicted draws due to class imbalance in training data, requiring threshold-based draw detection logic. Analysis revealed systematic home advantage overestimation—when GW29 featured 5 away wins versus 1 predicted, accuracy collapsed to 10%.

1.8 Dissertation Structure

The remainder of this dissertation is organized as follows:

Chapter 2: Methodology describes the software development approach, data collection strategy, machine learning methodology, technology selection rationale, and testing procedures.

Chapter 3: Technology Review surveys existing academic research on sports prediction, machine learning applications in football analytics, and describes the technologies used at a conceptual level.

Chapter 4: System Design presents the technical architecture including system diagrams, API endpoint specifications, database schema, and deployment configuration.

Chapter 5: System Evaluation analyzes prediction performance across all gameweeks with tables and graphs, explores feature importance rankings, and examines case studies of successful and failed predictions.

Chapter 6: Conclusion reflects on lessons learned, discusses fundamental limitations of data-driven football prediction, and proposes future improvements.

After reading this introduction, the reader should understand that this project built a complete, deployed football prediction system; evaluated its real-world performance over seven gameweeks; achieved 36.76% overall accuracy with a best gameweek of 60%, approaching academic benchmarks and providing valuable insights into ML limitations; and demonstrated both the potential and fundamental challenges of machine learning in highly unpredictable domains.

Chapter 2

Methodology

2.1 Development Approach

This project followed an incremental, iterative software development methodology. The system was built in stages, with each component developed, tested, and integrated before moving to the next. This approach allowed for continuous feedback and adjustment based on real-world performance.

Development began in January 2026 with research into football prediction techniques and available data sources. The first prototype—a command-line script training a Logistic Regression model—was completed in late January. Subsequent iterations added the Flask API (early February), PostgreSQL database (mid-February), React frontend (late February), and deployment infrastructure (early March). By March 2026, the system was generating predictions for live gameweeks.

Weekly supervisor meetings with Gerard Harrison provided structure and accountability. These 30-45 minute meetings followed a consistent format: review progress, demonstrate working functionality, discuss challenges, and establish goals for the coming week. Gerard's feedback influenced key decisions, including focusing on real-world validation rather than purely optimizing training accuracy, and comparing multiple ML algorithms rather than selecting one arbitrarily.

2.2 Requirements Gathering

Project requirements were established through academic guidelines, supervisor input, and analysis of existing football prediction systems. Requirements were captured as a feature checklist:

- Train ML model on historical Premier League data

- Generate predictions for upcoming gameweeks
- Display predictions via web interface
- Track accuracy against actual results
- Compare multiple ML algorithms
- Show live Premier League standings
- Deploy publicly accessible application

This lightweight approach suited a solo project with clear deliverables. GitHub served as the primary version control tool, with commits made incrementally after completing discrete units of work. The commit history serves as a development log tracking progress from initial prototype through final deployment.

2.3 Data Collection Strategy

The project required two categories of data: historical match results for training, and live league information for prediction generation.

2.3.1 Historical Match Data

Historical data began with a Kaggle dataset containing 12,441 Premier League matches from 1992 through the 2022-23 season. This base dataset was augmented using the football-data.org API. The `update_csv_with_current_season.py` script fetches finished matches from the API—which provides access to historical data from recent seasons. Running this script in early 2026 fetched all matches from the 2023-24, 2024-25, and 2025-26 seasons (through March 2026), adding approximately 1,140 matches.

For training, only matches from these last three seasons were used, totaling 2,261 matches. This temporal restriction ensures training data reflects modern football—tactical trends and team compositions from earlier eras may not generalize to current conditions. This approach of using historical match data for training is common in sports prediction research [6, 7]. After each gameweek during evaluation, the script was re-run to append newly finished matches.

2.3.2 Live League Data

Prediction generation requires current league standings. The football-data.org API provides RESTful endpoints for live Premier League tables, including team names,

positions, points, and goal statistics. An API key (obtained via free registration) authenticates requests.

The `services/standings_service.py` module fetches standings before each prediction run and caches results for one hour to minimize API calls. Rate limiting posed a constraint—the free tier allows 10 requests per minute, requiring retry logic with exponential backoff during testing.

2.4 Machine Learning Methodology

The ML pipeline follows standard supervised learning practices: split data into training and testing sets, train multiple models, evaluate performance, select the best model, and deploy for inference.

2.4.1 Train-Test Split

The 2,261 historical matches were split using a temporal approach: 628 matches from seasons 2023-24 and 2024-25 for training, and 1,411 matches from the 2025-26 season for testing. Training was restricted to these two completed seasons to ensure the model learned from recent tactical patterns, with the entire 2025-26 season reserved for real-world evaluation. This temporal split ensures the model is evaluated on genuinely unseen future data, reflecting real-world deployment conditions more accurately than a random split would.

2.4.2 Model Selection

Three algorithms were trained and compared: **Logistic Regression:** A linear classifier [4] modeling probability of each outcome. Provides interpretable coefficients.

Random Forest: An ensemble method [5] training 200 trees. Handles non-linear relationships but less interpretable.

Decision Tree: A single tree recursively partitioning feature space. Highly interpretable but prone to overfitting.

All models used scikit-learn [8] implementations with default hyperparameters. No grid search or hyperparameter tuning was performed.

2.4.3 Feature Engineering

Feature engineering transformed raw match data into numerical features. The process operated in two stages:

Historical features computed by scanning prior matches:

- **Form:** Sum of points (3 for win, 1 for draw, 0 for loss) from last 5 games
- **Goals:** Average goals scored and conceded over all historical matches
- **Draw rate:** Percentage of last 5 matches ending in draws
- **Form-based position:** Team ranking based on recent form points

Live features fetched from API before prediction:

- **Actual league position:** Current standing (1-20)
- **Position gap:** Difference between home and away positions

Feature engineering is critical in sports prediction, as raw match data provides limited signal without domain-specific transformations [9]. Feature importance analysis (Chapter 5) revealed that positional features overall contributed most to prediction accuracy, with both form-based rankings and actual API positions among the top features.

2.5 Technology Selection

Technology choices were driven by ease of development, ML library availability, and deployment constraints.

Python was selected for backend development due to its dominant position in machine learning. The scikit-learn library [8] provides battle-tested implementations of Logistic Regression, Random Forest, and Decision Tree. Python's pandas library simplifies CSV manipulation and data preprocessing.

Flask [10] was chosen over heavier frameworks like Django because the project requires RESTful API endpoints rather than server-rendered web pages. Flask's minimalist design suited rapid prototyping.

React [11] was selected for the frontend to demonstrate modern JavaScript skills relevant to industry positions. React's component-based architecture cleanly separates concerns (Dashboard, History, Models, TeamStats pages). Tailwind CSS provided utility classes for rapid styling without writing custom CSS.

PostgreSQL [12] was chosen for data persistence over NoSQL alternatives like MongoDB because match predictions have a well-defined schema (home team, away team, probabilities, results) suited to relational tables. PostgreSQL's ACID guarantees ensure predictions and results remain consistent.

Render.com was selected for deployment as it offers free-tier hosting with GitHub integration for continuous deployment. Render's managed PostgreSQL service ($7/month) ensures data persistence beyond free tier limits.

2.6 Testing and Validation

Testing occurred at component and system levels. Component testing was informal and manual—API endpoints were tested via `curl`, database operations verified by querying PostgreSQL directly, and frontend components tested in the browser during development.

No automated unit testing framework was employed. For a solo project with a small codebase developed iteratively, the overhead exceeded the benefit. Manual testing at each development stage caught bugs before they propagated.

System-level validation occurred through real-world deployment. Starting with Gameweek 27, the system generated predictions for upcoming matches, which were compared against actual results. The `services/compare_results.py` script automated this comparison: fetch finished matches, match them to stored predictions, mark each as correct or incorrect, and update the database.

The `services/evaluation_service.py` script calculated overall accuracy and per-gameweek breakdowns.

This empirical approach—deploying and evaluating real-world performance—provides stronger validation than cross-validation on historical data alone. It exposes the model to distribution shifts, data quality issues, and operational challenges that controlled testing cannot reveal.

2.7 Problem-Solving Approach

Several technical challenges emerged during development:

Draw Prediction Failure: The initial model never predicted draws. Investigation revealed class imbalance: the training set contained approximately 45% home wins, 30% away wins, and 25% draws. Class imbalance is a well-documented challenge in machine learning classification tasks [13]. Logistic regression, optimizing for overall accuracy, learned to ignore the minority class. The solution involved adding threshold logic: if `max(home_prob, draw_prob, away_prob) < 0.4`, classify as draw.

Team Name Inconsistencies: The Kaggle CSV used names like "Man City" while the API returned "Manchester City FC." A normalization dictionary mapping alternative names to canonical forms resolved mismatches.

Home Advantage Overestimation: Gameweek 29 accuracy collapsed to 10% when 5 matches resulted in away wins versus 1 predicted. Analysis revealed systematic bias toward home teams, likely stemming from historical home advantage in training data not reflecting modern conditions.

These problems were identified through error analysis—examining which predictions failed and why—rather than automated testing. The empirical approach

of deploying early, gathering feedback, and iterating based on observed failures proved effective.

2.8 Summary

This chapter described the methodology employed to build and evaluate the football prediction system. An incremental development approach with weekly supervisor meetings provided structure. Data was collected from Kaggle and the football-data.org API. Machine learning followed standard supervised practices with train/test split, algorithm comparison, and feature engineering. Technology selection prioritized practical considerations. Validation occurred through real-world testing across seven gameweeks, exposing the system to conditions laboratory testing cannot simulate.

Chapter 3

Technology Review

This chapter surveys existing research on football prediction, machine learning techniques for classification, and the technologies employed in this project. The review establishes theoretical foundations for design decisions made in later chapters and demonstrates that the project builds upon established academic and industry practices rather than operating in isolation.

3.1 Football Prediction Research

Football match outcome prediction has been studied extensively with both academic and commercial contexts. The problem attracts interest because it combines accessible public data, clear success metrics (prediction accuracy), and genuine difficulty–even sophisticated models struggle to exceed 55% accuracy [1]. This ceiling exists because football outcomes depend not only on quantifiable team strength but also on dramatic events: injuries during play, referee decisions, momentary lapses in concentration, and sheer luck. Constantinou and Fenton [1] identified a fundamental issue with early prediction models: inadequate scoring rules that rewarded overconfident predictions rather than well-calibrated probability estimates. Their work emphasized that prediction systems should output probability distributions (e.g., 40% home win, 30% draw, 30% away win) rather than binary classifications,allowing users to access model confidence.This insight influenced the design of this projects prediction system,which stores and displays probability distributions for all three outcomes rather than merely stating a winner. Baboota and Kaur [2] applied machine learning to English Premier League prediction, testing multiple algorithms including Logistic Regression,Random Forest and Gradient Boosting.They achieved approximately 50% accuracy on test data, consistent with academic benchmarks.Their feature engineering approach-combining team form, league position and goal statistics-demonstrated that relatively simple

features can capture most predictive signal.Complex features derived from player-level statistics or tactical formations provided marginal improvements at significant computational cost.This finding validated the decision to focus on team-level aggregates (form, goals,positions) rather than attempting granular player modeling. Tax and Joustra [3] predicted Dutch football result using publicly available data, explicitly comparing their approach to commercial bookmaker models.They found that models trained on public data plateaued around 45-50% accuracy, while bookmakers (with access to proprietary information such as confirmed lineups and injury reports) reached 52-55%.This 5-10% gap quantifies the value of insider information and sets realistic expectations for student projects restricted to public data sources.The current project targets the 40-50% academic benchmark rather than attempting to match commercial systems operating with informational advantages. Joseph et al. [6] applied Bayesian networks to football prediction,modeling conditional dependencies between variables (e.g.home advantage depends on crowd size,which varies by team).Bayesian approaches excel at representing uncertainty and incorporating domain knowledge through prior distributions.However, they require careful specification of network structure and priors-knowledge often unavailable to student researchers without extensive domain expertise.Logistic Regression and Random Forest , in contrast,learn relationships directly from data without requiring explicit dependency modeling,making them more suitable for exploratory projects. Bunker and Thabtah [7] surveyed machine learning frameworks for sports prediction across multiple domains (football, basketball, cricket). They identified common challenges:class imbalance (home wins more frequent than away wins), feature selection (which statistics matter most), and temporal drift(team strength changes throughout a season). Their review concluded that no single algorithm dominates-Logistic Regression,Random Forest, and Support Vector Machines achieve similar performance when features are well-engineered.This finding justified comparing multiple algorithms in this project rather than selecting one arbitrarily. A consistent theme across football match prediction is the importance of feature engineering.Raw data (team names,match dates,scores) provides minimal predictive power.Transforming this data into meaningful features-recent form, positional strength,home advantage proxies-creates signal for machine learning models to exploit. The next section examines learning algorithms used for classification tasks,followed by a detailed discussion of feature engineering techniques.

3.2 Machine Learning for Classification

Classification is a supervised learning task where models learn to assign inputs (e.g.,upcoming matches) to discrete categories(home win,draw,away win) based

on labeled training examples.This section reviews three algorithms employed in this project: Logistic Regression,Random Forest,and Decision Tree.

3.2.1 Logistic Regression

Logistic Regression [4] is a linear model for binary and multi-class classification.Despite its name, it performs classification rather than regression by modeling the log-odds(logit) of class membership as a linear function of features. For multi-class problems like football prediction (three outcomes: home/draw/away),multi-class logistic regression extends the binary case using a softmax function to produce probability distributions over classes.The model assumes that the log-odds outcome k occurring is a linear combination of features.

$$\log\left(\frac{P(Y=k|X)}{P(Y=\text{baseline}|X)}\right) = \beta_0^{(k)} + \beta_1^{(k)}X_1 + \beta_2^{(k)}X_2 + \cdots + \beta_p^{(k)}X_p \tag{3.1}$$

where $X_1, \ldots, X_p$ are features(form,goals,positions), β coefficients are learned during training, and the baseline class is typically the most frequent outcome (home win). Training minimizes a loss function (cross-entropy) using gradient descent or Newton's method. Logistic Regression's primary advantage is interpretability.The coefficients β directly indicate feature importance: large positive coefficients increase the probability of that outcome,while negative coefficients decrease it.In this project,examining coefficients reveal that positional features overall contributed most to predictions, with form-based rankings and actual API positions both among the top features. This interpretability aids in debugging and builds trust in model predictions.However,Logistic Regression assumes linear relationships between features and log-odds.If the true relationship is non-linear(e.g., form matters more for mid-table team than for champions), the model cannot capture it without explicit feature engineering(creating interaction terms, polynomial features).This limitation motivates comparing against non-linear algorithms like Random Forest.

3.2.2 Random Forest

Random Forest [5] is an ensemble learning method that constructs multiple decision trees during training and outputs the mode (for classification) or mean (for regression) of individual tree predictions.Each tree is trained on a bootstrapped sample of the training data,and at each node split,a random subset of features is considered rather than all features.This randomness de-correlates trees,reducing overfitting compared to a single deep decision tree. For classification, the Random Forest prediction is:

$$\hat{Y} = \text{mode}\{T_1(X), T_2(X), \ldots, T_B(X)\} \tag{3.2}$$

where T_i represents the i-th tree's prediction and B is the number of trees (200 in this project). Random Forests handle non-linear relationships and feature interactions naturally .If home advantage matters more when the home team is higher, the forest will learn this interaction without explicit feature engineering.They are also robust to noisy features-irrelevant variables are unlikely to be selected for splits,reducing their impact on predictions. The primary disadvantage is reduced interpretability.While individual trees are interpretable (each node split represents a decision rule), averaging 200 trees obscures the logic.Feature importance can be computed by measuring how much each feature reduces impurity (e.g., Gini index) across all tress ,but this provides less direct insight than Logistic Regression coefficients.Additionally,Random Forests can overfit small datasets if trees are allowed to grow deep, though this risk is mitigated by limiting tree depth and requiring minimum samples per leaf.In this project, Random Forest achieved 49.33% training accuracy, under-performing Logistic Regression's 59.74%.This suggests that well-engineered linear features (form, position, goals) capture most predictive signal, and most additional non-linear interactions provide limited benefit.Alternatively, the 628-match training set may be too small for Random Forest to effectively learn complex patterns-ensemble methods typically shine with large datasets.

3.2.3 Decision Tree

Decision Trees recursively partition the feature space into regions,assigning each region a class label based on majority vote of training examples in that region.At each node,the algorithm selects the feature and threshold that best splits the data according to an impurity measure (Gini index or entropy) A simplified decision tree for football prediction might look like:

```
if HomePosition < 5:
    predict Home Win
else:
    if AwayForm > 10:
        predict Away Win
    else:
        predict Draw
```

Decision Trees are highly interpretable-the entire prediction logic can be visualized as a flowchart.This transparency aids debugging and explanation to the non-technical stakeholders.However,a single trees are prone to overfitting: they can memorize training data by creating deep,complex trees with nodes matching individual training examples.This overfitting is evident in this project's results :the Decision Tree achieved only 37.07% accuracy, the worst of the three algorithms.Techniques to mitigate overfitting include pruning(removing branches that

provide little predictive value),limiting tree depth,and requiring minimum samples per leaf.Random Forests address overfitting by averaging many trees,trading interpretability for improved generalization.

3.2.4 Class Imbalance

A challenge shared across all three algorithms is class imbalance [13]: the training data contains approximately 45% home wins, 30% away wins,and 25% draws.Standard classification algorithms optimize overall accuracy,which can be maximized by always predicting the majority class (home win).A model that predicts "home win" for every match achieves 45% accuracy without learning anything meaningful. Techniques to address class imbalance include:

- **Resampling:** Over-sample minority classes (draws) or under-sample majority classes (home wins) to balance training data
- **Class weights:** Penalize misclassifications of minority classes more heavily during training
- **Threshold adjustment:** Modify decision thresholds to increase sensitivity to minority classes

This project employed threshold adjustment:if the maximum predicted probability fell below 40%, the prediction was overridden to "draw".This improved draw detection but remained imperfect-Gameweek 28 predicted 3 draws,all of which proved incorrect.More sophisticated approaches such as SMOTE(Synthetic Minority Over-Sampling Technique) or ensemble methods specifically designed for imbalanced data might improve draw prediction in future iterations.

3.3 Feature Engineering in Sports Prediction

Feature engineering transforms raw data into representations suitable for machine learning [9].For football prediction,raw data consists of team names,match dates, and scores.These categorical and temporal variables must be converted into numerical features capturing relevant patterns.

3.3.1 Form Metrics

Team form quantifies recent performance,typically measured as points accumulated over the last N matches (where $N = 5$ in this project). A win awards 3

points,a draw 1 point, and a loss 0 points.Form features for an upcoming match between Team A (home) and Team B (away) are:

$$\text{HomeForm} = \sum_{i=1}^{5} \text{points(Team A's last 5 matches)} \tag{3.3}$$

$$\text{AwayForm} = \sum_{i=1}^{5} \text{points(Team B's last 5 matches)} \tag{3.4}$$

Form captures momentum-teams on winning streaks may carry confidence and tactical cohesion into upcoming matches.However,form has limitations.A team's last 5 matches may include fixtures against both top-tier and bottom-tier opponents,conflating difficulty.A team that defeated the league leader and drew with mid-table teams earns the same 4 points as a team that defeated two relegation-threatened sides.More sophisticated form metrics weight recent matches by opponent strength,though this introduces circularity(opponent strength itself must be estimated). Offensive and defensive strength are captured through goal statistics:

$$\text{HomeGoalsAvg} = \frac{\sum \text{goals scored by Team A}}{\text{number of matches played}} \tag{3.5}$$

$$\text{HomeConcededAvg} = \frac{\sum \text{goals conceded by Team A}}{\text{number of matches played}} \tag{3.6}$$

These averages are calculated over all historical matches in the dataset. Teams with high goals-scored averages are strong offensively; teams with low goals-conceded averages have solid defenses.Combining offensive and defensive metrics for both home and away team creates features that proxy overall team quality. A limitation is that averages smooth over temporal variation . A team may have a high season-long goals average due to strong early-season performance but have declined recently.Weighting recent matches more heavily (exponential decay) can address this,though it adds complexity.

3.3.2 Positional Features

League position (1st through 20th in the Premier League) serves as a proxy for overall team strength.Two position features were engineered: **Form-based position:**Rank teams by recent form(points from the last 5 matches).Teams with the highest form achieve position 1,next-highest position 2 ,etc. **Actual league position:** Fetch current standing from the football-data.org API .This reflects cumulative season-long performance rather than recent form alone. Feature importance

analysis (Chapter 5) revealed that positional features as a whole dominated prediction accuracy, with form-based rankings contributing slightly more than API positions. Nevertheless, integrating real-time API data provided additional signal beyond calculated proxies alone. A team ranked 3rd has likely demonstrated consistent quality over 25+ matches, complementing the shorter-term signal captured by form-based rankings.

3.3.3 Draw Rate Features

Draws represent approximately 25% of Premier League matches, and predicting them is notoriously difficult.The model initially never predicted draws,always defaulting to home or away wins.To address this,draw-specific features were added:

$$\text{HomeDrawRate} = \frac{\text{draws in Team A's last 5 matches}}{5} \tag{3.7}$$

These features help identify teams with defensive tactics or low-scoring tendencies that often result in draws.However,draw prediction remain challenging-—the model's accuracy on draws (25%) approximately matched random guessing.

3.3.4 Home Advantage

Home advantage-the tendency for home teams to win more frequently than away team-—is well-documented in sports research [14, 15]. Proposed explanations include crowd support,familiarity with playing conditions,reduced travel fatigue, and referee bias (consciously or unconsciously favoring the home team). Pollard [14] surveyed home advantage across multiple leagues and eras,finding that Premier League home teams win approximately 45-48% of matches compared to 25-28% for away teams (the remainder being draws). This imbalance persisted across decades, suggesting a stable phenomenon.Courneya and Carron [15] analyzed psychological mechanisms underlying home advantage,proposing that crowd support increase player motivation and arousal.Large,vocal crowds create psychological pressure on referees,potentially influencing marginal decisions (penalty calls,yellow cards).However,home advantage may be diminishing in modern football.COVID-19 lockdown forced matches to be played in empty stadiums, and subsequent analysis found reduced home win rates,supporting the crowd-based explanation.Even post-pandemic,evolving tactical approaches and increased professionalism may erode home advantage.This project did not explicitly create a "home advantage" feature-instead, the home/away is implicit in separate form and goal features for home versus away teams.Analysis in Chapter 5 reveals the model systematically over-predicted home wins, suggesting it inherited historical home advantage bias from training data that no longer reflects current conditions.

3.4 Backend Technologies

This section reviews the technologies used to implement the backend: Python for scripting and machine learning,Flask for API development , and PostgreSQL for data persistence.

3.4.1 Python and scikit-learn

Python [16] is a high-level,interpreted programming language that become dominant in data science and machine learning due to its extensive ecosystem of libraries.The scikit-learn library [8] provides unified interfaces for dozens of machine learning algorithms,data preprocessing utilities, and model evaluation tools.Scikit-learns design philosophy emphasizes ease of use and consistency.All estimators (models) implement a `fit(X, y)` method for training and a `predict(X)` method for inference.This uniformity allows experimenting with different algorithms by changing a single line of code — replacing `LogisticRegression()` with `RandomForestClassifie` requires no changes to data preprocessing or evaluation pipelines. For this project,scikit-learn simplified model training to a few lines:

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
accuracy = model.score(X_test, y_test)
```

The pandas library [17] provided DataFrames for CSV manipulation, enabling filtering, aggregation, and feature calculation with readable, concise syntax. Without pandas, implementing feature engineering would require explicit loops and manual indexing—error-prone and difficult to debug.

3.4.2 Flask and RESTful APIs

Flask [10] is a micro web framework for Python.Unlike full feature frameworks such as Django that include ORM,authentication,and template engines,Flask provides minimal core functionality:HTTP routing,request/response handing,and JSON serialization.Developers add capabilities as needed through extensions,keeping the codebase lightweight. For this project, Flask exposed five RESTful API endpoints:

- `GET /api/predictions` - Retrieve upcoming match predictions
- `GET /api/history` - Fetch past predictions with actual results
- `GET /api/models` - Compare algorithm accuracies

- `GET /api/accuracy` - Overall prediction accuracy
- `GET /api/standings` - Live Premier League table

RESTful API design [18] follows principles of statelessness(each request contains all information to process it),resource-based URLS(endpoints represent resources like "predictions" or "standings"), and standard HTTP methods(GET for retrieval,POST for creation,PUT for update,DELETE for removal).This project uses only GET requests as the frontend retrieves data without modifying server state directly.Flasks simplicity suited rapid prototyping.The entire API implementation total approximately 150 lines of Python,far less than equivalent implementations in enterprise frameworks requiring configuration files, dependency injection, and boilerplate code.

3.4.3 PostgreSQL

PostgreSQL [12, 19] is an open-source relational database management system emphasizing standards compliance,extensibility, and ACID (Atomicity, Consistency, Isolation, Durability) guarantees.ACID properties ensure that database transactions either complete fully or have no effect,preventing data corruption during crashes or concurrent access. For this project, PostgreSQL stores predictions in a relational schema:

```
CREATE TABLE predictions (
    id SERIAL PRIMARY KEY,
    home_team VARCHAR(50),
    away_team VARCHAR(50),
    prediction VARCHAR(20),
    home_win_prob FLOAT,
    draw_prob FLOAT,
    away_win_prob FLOAT,
    actual_result VARCHAR(20),
    correct BOOLEAN,
    gameweek INTEGER
);
```

This schema enables queries like "find all Gameweek 29 predictions" or "calculate accuracy for matches where home_win_prob > 0.6." Relational databases excel at structured queries with filtering, aggregation, and joins—capabilities that NoSQL document stores lack or provide inefficiently.

PostgreSQL's managed service on Render.com handles backups, replication, and performance tuning automatically, allowing focus on application logic rather than database administration.

3.5 Frontend Technologies

The frontend is built with React [11],a JavaScript library for building user interfaces.Reacts component-based architecture encourages breaking UIs into reusable,composable pieces. Each component manages its own state and rendering logic, promoting modularity and testability. For this project, the UI consists of four main components:

- **Dashboard:** Displays upcoming predictions and overall accuracy
- **History:** Shows past predictions grouped by gameweek, color-coded by correctness
- **ModelComparison:** Compares Logistic Regression, Random Forest, and Decision Tree accuracies
- **TeamStats:** Presents the live Premier League table with visual highlighting

React components fetch data from the Flask API using Axios, an HTTP client library. When a component mounts, it issues a GET request to the appropriate endpoint, updates its state with the response, and re-renders to display the data. This separation between frontend (presentation) and backend (business logic and data) follows the single-page application (SPA) architecture, where the frontend is a static JavaScript bundle that communicates with the backend via APIs.

Tailwind CSS provided utility classes for styling (e.g., `text-blue-500`, `p-4`, `rounded-lg`) without writing custom CSS. This approach accelerates development by eliminating the need to name classes and maintain separate stylesheet files. However, it produces verbose HTML with many class names—a trade-off between development speed and markup cleanliness.

React's primary advantage is declarative rendering: developers describe what the UI should look like for a given state, and React handles DOM updates efficiently. This contrasts with imperative approaches (e.g., jQuery) where developers manually manipulate DOM elements. React's virtual DOM diffing algorithm minimizes expensive browser reflows, improving performance for dynamic UIs.

3.6 Deployment and Cloud Services

The system is deployed on Render.com, a platform-as-a-service (PaaS) that abstracts infrastructure management. PaaS platforms sit between Infrastructure-as-a-Service (IaaS) offerings like AWS EC2 (where users manage virtual machines, operating systems, and software stacks) and Software-as-a-Service (SaaS) products (fully managed applications like Gmail).

Render provides:

- **Automatic deployment:** Pushing to the `main` branch on GitHub triggers builds and deployments
- **Managed databases:** PostgreSQL runs on Render's infrastructure with automated backups
- **Free tier hosting:** Static sites and web services have free tiers suitable for student projects
- **Environment variables:** Secrets like database URLs and API keys are configured through a dashboard

The backend runs as a Gunicorn-served Flask application. Gunicorn is a WSGI (Web Server Gateway Interface) server that handles HTTP requests and forwards them to the Flask app. Gunicorn manages multiple worker processes for concurrency, though the free tier's limited resources constrain throughput.

The frontend builds to static files (HTML, CSS, JavaScript) served via Render's content delivery network (CDN). CDNs distribute content across geographically distributed servers, reducing latency by serving users from nearby locations. For a student project with limited traffic, CDN benefits are minimal, but the architecture demonstrates production-ready deployment patterns.

Continuous deployment enables rapid iteration: fixing a bug or adding a feature requires only committing code to GitHub—Render automatically rebuilds and deploys within minutes. This automation reduces friction compared to manual deployment processes involving SSH, file transfers, and server restarts.

3.7 Summary

This chapter surveyed the theoretical and practical foundations underlying the football prediction system. Football prediction research [1, 2, 3, 6, 7] established that 40-50% accuracy is typical for academic models using public data, and that feature engineering is more impactful than algorithm selection. Machine learning techniques—Logistic Regression [4], Random Forest [5], and Decision Trees—provide different trade-offs between interpretability and capacity for non-linear relationships. Feature engineering transforms raw match data into predictive signals, with positional features overall proving most informative, combining form-based rankings and actual API positions. Backend technologies (Python [16], scikit-learn [8], Flask [10], PostgreSQL [12]) enabled rapid prototyping and deployment. Frontend technologies (React [11], Tailwind CSS) created an intuitive user

interface. Cloud deployment on Render.com provided automated, production-ready hosting. The next chapter describes how these technologies were integrated into a coherent system architecture.

Chapter 4

System Design

This chapter presents the technical architecture of the football prediction system,explaining how components integrate to form a coherent whole.System design decisions were informed by the technology review (Chapter 3) and constrained by practical considerations including development time,development cost and the need for real-time data integration.

4.1 System Architecture Overview

The system follows a three-tier architecture separating presentation (React frontend),application logic(Flask backend), and data storage(PostgreSQL database).This separation enables independent development,testing,and scaling of each tier.External data integration occurs through the football-data.org RESTful API,which provides live league standings and match results. Figure 4.1 illustrates the high-level architecture and data flow between components.

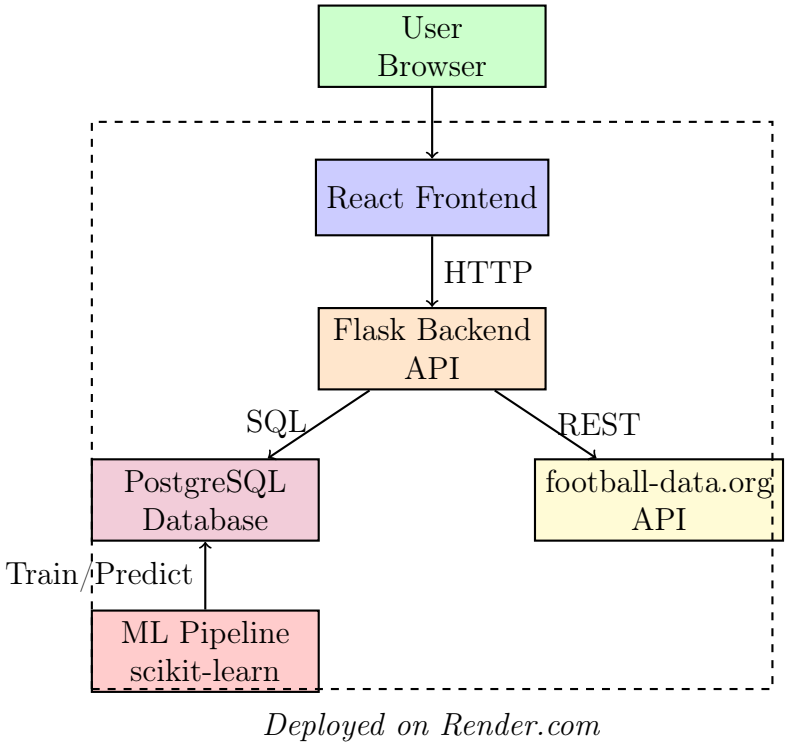


Figure 4.1: System Architecture Overview showing data flow between components

The frontend communicates with the backend exclusively through HTTP requests returning JSON responses.This RESTful design enables the frontend and backend to evolve independently-changes to the database schema or ML algorithms require no frontend modifications as API contracts remain stable.The backend serves dual purposes: responding to API requests from the frontend,and executing batch processes (model training,prediction generation) triggered manually via command-line scripts.This design separates user-facing services (always available) from computationally intensive tasks (run periodically).External data integration occurs at two points.During prediction generation,the backend fetches current league standings from football-data.org to populate positional features.After each gameweek concludes, a batch script fetches match results to compare against scored predictions and update the database. All components deploy on Render.coms platform as a service infrastructure.The frontend builds to static files served via CDN,the backend runs as a Gunicorn-served Flask application,and the database operates on Renders managed PostgreSql service.Github integration enables continuous deployment: pushing to the main branch triggers automatic rebuilds.

4.2 Backend API Design

The Flask backend exposes five RESTful endpoints providing access to predictions,historical results,model statistics,and live league data.All endpoints use HTTP GET methods and return JSON-formatted responses.

4.2.1 API Endpoint Specifications

GET /api/predictions Returns upcoming match predictions for the next gameweek.Each prediction includes home team, away team, predicted outcome, probability distribution, and match date.

Response format:

```
{
  "predictions": [
    {
      "home_team": "Arsenal",
      "away_team": "Chelsea",
      "prediction": "Home Win",
      "home_win_prob": 0.62,
      "draw_prob": 0.23,
      "away_win_prob": 0.15,
      "date": "2026-03-22 15:00:00",
      "gameweek": 32
    },
    ...
  ]
}
```

GET /api/history?limit=N Return past prediction with actual results,optionally limited to the most recent N predictions.Each record includes whether the prediction was correct.

Response format:

```
{
  "history": [
    {
      "home_team": "Liverpool",
      "away_team": "Man City",
      "prediction": "Away Win",
      "actual_result": "Draw",
      "correct": false,
```

```
      "gameweek": 31,
      "date": "2026-03-15 17:30:00"
    },
    ...
  ]
}
```

GET /api/models Returns training accuracy for all three algorithms,enabling frontend comparison.

Response format:

```
{
  "logistic_regression": 0.5974,
  "random_forest": 0.4933,
  "decision_tree": 0.3707
}
```

GET /api/accuracy Returns overall prediction accuracy across all evaluated gameweeks.

Response format:

```
{
 "accuracy": 0.3676,
"correct": 25,
"total": 68
}
```

GET /api/standings

Returns the current Premier League table with all 20 teams,fetched in real-time from the football-data.org API.

Response format:

```
{
  "standings": [
    {
      "position": 1,
      "team": "Arsenal",
      "points": 73,
      "wins": 22,
      "draws": 7,
      "losses": 2,
      "goals_for": 68,
      "goals_against": 24
```

```
    },
    ...
  ]
}
```

4.2.2 Error Handling

All endpoints return HTTP 500 status codes with error messages if database queries fail or external API requests timeout.The frontend displays user-friendly error messages ("Unable to load predictions.Please try again later.") rather than exposing technical details. Rate limiting on the football-data.org API (10 requests/minute) occasionally triggers errors during deployment.The production system mitigates this through caching:standings are cached for one hour,reducing API calls when multiple users access the frontend simultaneously.

4.3 Database Schema

Match predictions and results are stored in a single PostgreSQL table optimized for common query patterns: retrieving predictions for a specific gameweek, calculating overall accuracy, and filtering by correctness.

Figure 4.2 illustrates the predictions table structure.

Column	Type	Description
id	SERIAL	Primary key, auto-incrementing
home_team	VARCHAR(50)	Home team name
away_team	VARCHAR(50)	Away team name
prediction	VARCHAR(20)	Predicted outcome: "Home Win", "Draw", or "Away Win"
home_win_prob	FLOAT	Probability of home win (0.0-1.0)
draw_prob	FLOAT	Probability of draw (0.0-1.0)
away_win_prob	FLOAT	Probability of away win (0.0-1.0)
match_date	TIMESTAMP	UTC timestamp of match kickoff
gameweek	INTEGER	Premier League gameweek number (1-38)
actual_result	VARCHAR(20)	Actual outcome after match concludes (NULL until result known)
correct	BOOLEAN	Whether prediction matched actual result (NULL until result known)
home_goals	INTEGER	Goals scored by home team (NULL until result known)
away_goals	INTEGER	Goals scored by away team (NULL until result known)
created_at	TIMESTAMP	When prediction was generated

Figure 4.2: Predictions Table Schema

The schema supports the following queries efficiently.

- **Retrieve upcoming predictions:**
  `SELECT * FROM predictions WHERE actual_result IS NULL ORDER BY match_date;`

- **Calculate gameweek accuracy:**
  ```
  SELECT COUNT(*) FILTER (WHERE correct = true) / COUNT(*)
  FROM predictions WHERE gameweek = 31;
  ```

- **Find all incorrect predictions:**
  `SELECT * FROM predictions WHERE correct = false;`

No foreign keys or joins are required—the table is self-contained. Indexing on the gameweek column accelerates queries filtering by specific gameweeks, though with only 68 total predictions, performance differences are negligible.

4.4 Feature Engineering Workflow

Feature engineering transforms raw match data (team names,dates) into 16 numerical features suitable for machine learning.The process operates in two stages: calculating historical aggregates from the CSV dataset, and fetching live league positions from the API.

Figure 4.3 illustrates the feature engineering workflow.

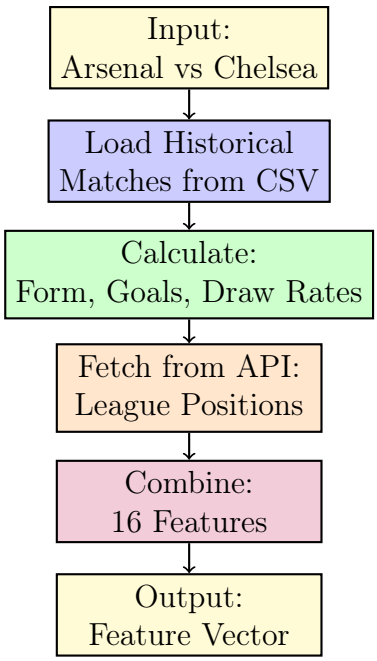


Figure 4.3: Feature Engineering Workflow

The historical aggregation step scans all prior matches involving each team,computing:

- **Form:** Points from last 5 matches (3 for win, 1 for draw, 0 for loss)
- **Goals:** Average goals scored and conceded across all historical matches
- **Draw rates:** Percentage of last 5 matches ending in draws
- **Form-based position:** Team ranking by recent form points

The API integration step fetches current Premier League standings,extracting:

- **Actual league position:** 1-20 ranking in the table
- **Position gap:** Difference between home and away positions

These 16 features feed directly into the trained Logistic Regression model, which outputs probability distributions for home win, draw, and away win. The highest probability determines the prediction stored in the database.

4.5 Frontend Component Structure

The React frontend consists of four main pages,each implemented as a functional component:

4.5.1 Dashboard Component

Displays upcoming predictions,overall accuracy statistics,and model training accuracy. Fetches data from `/api/predictions`, `/api/accuracy`, and `/api/models` on component mount.Updates automatically when new predictions are generated.

4.5.2 History Component

Shows past predictions grouped by gameweek with color-coded correctness (green for correct, red for incorrect). Fetches data from `/api/history` with no limit, returning all evaluated predictions. Users can visually scan for patterns in incorrect predictions.

4.5.3 Model Comparison Component

Compares training accuracies of Logistic Regression (59.74%), Random Forest (49.33%), and Decision Tree (37.07%). Also lists all 16 features ordered by importance (measure by Logistic Regression coefficients).Fetches data from `/api/models`.

4.5.4 Team Stats Component

Presents the live Premier League table with visual highlighting:green background for top 4 (Champions League Qualification),red background for bottom 3 (relegation zone).Fetches data from `/api/standings`, which proxies the football-data.org API. All components use React hooks (`useState`, `useEffect`) for state management and side effects.Axios handles HTTP requests with error handling for network failures.Tailwind CSS utility classes provide responsive styling without custom CSS files.

4.6 User Interface Screenshots

The following screenshots demonstrate the deployed system's user interface. All images are from the live application at `https://fyp-frontend-wgcl.onrender.com`.

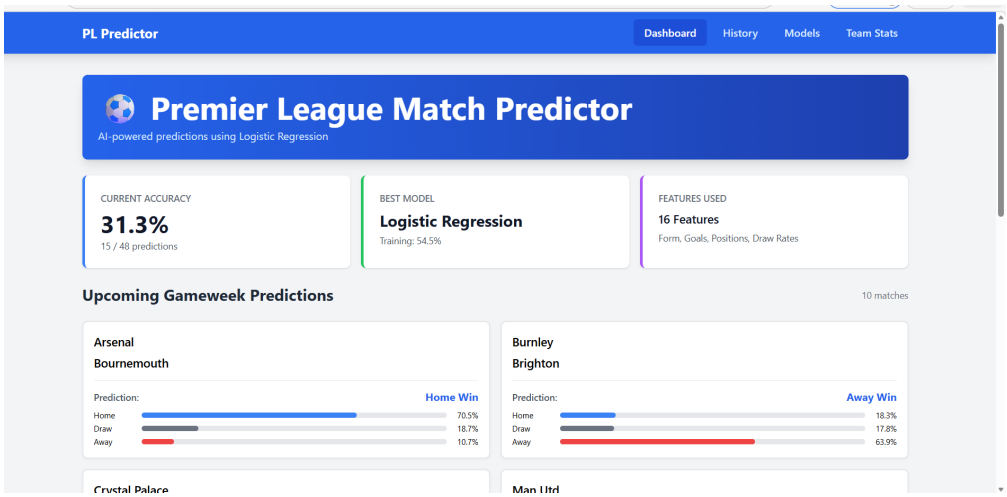


Figure 4.4: Dashboard displaying upcoming predictions and accuracy statistics

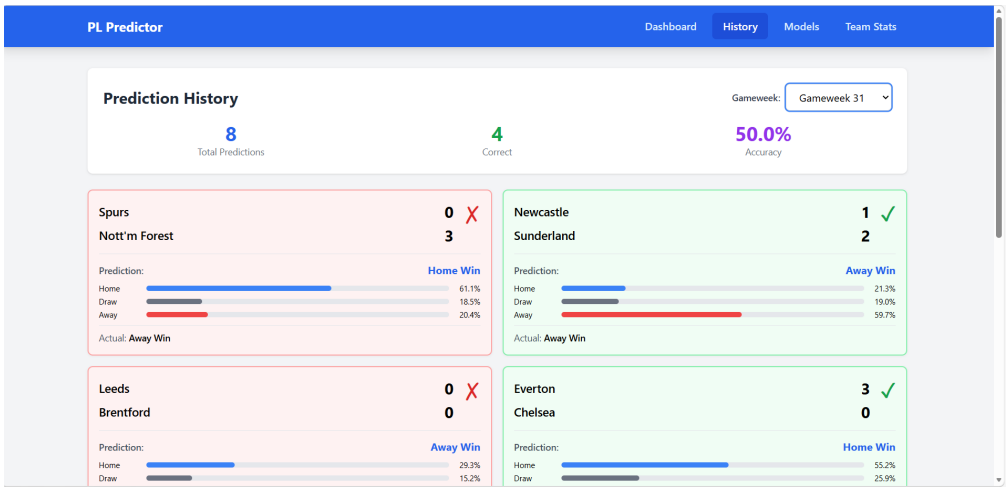


Figure 4.5: History page showing past predictions color-coded by correctness

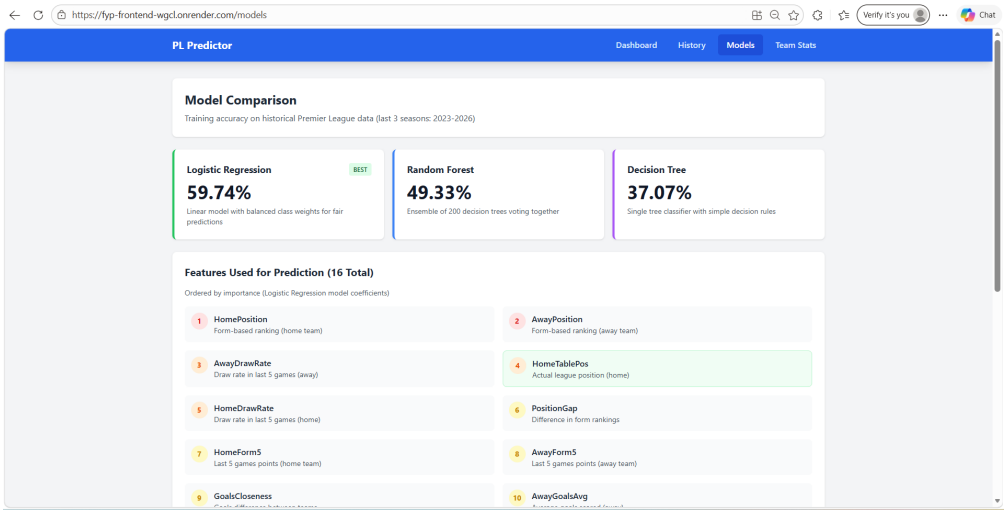
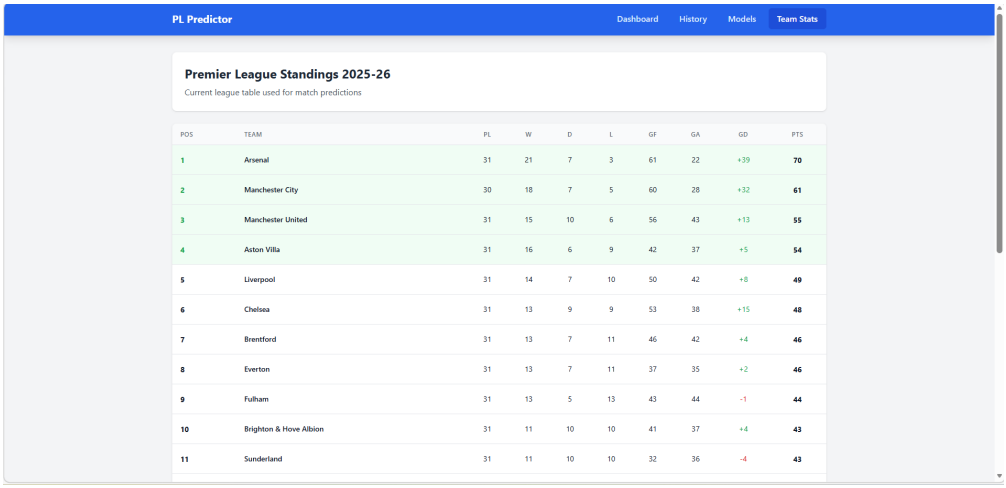


Figure 4.6: Model comparison page listing algorithms and feature importance



POS	TEAM	PL	W	D	L	GF	GA	GD	PTS
1	Arsenal	31	21	7	3	61	22	+39	70
2	Manchester City	30	18	7	5	60	28	+32	61
3	Manchester United	31	15	10	6	56	43	+13	55
4	Aston Villa	31	16	6	9	42	37	+5	54
5	Liverpool	31	14	7	10	50	42	+8	49
6	Chelsea	31	13	9	9	53	38	+15	48
7	Brentford	31	13	7	11	46	42	+4	46
8	Everton	31	13	7	11	37	35	+2	46
9	Fulham	31	13	5	13	43	44	-1	44
10	Brighton & Hove Albion	31	11	10	10	41	37	+4	43
11	Sunderland	31	11	10	10	32	36	-4	43

Figure 4.7: Team Stats page displaying live Premier League standings

The user interface prioritizes clarity and simplicity. Color-coding (green for correct predictions, red for incorrect) enables quick visual scanning. Responsive design ensures the application functions on mobile devices, tablets, and desktop computers without layout issues.

4.7 Deployment Configuration

The system deploys on Render.com using the following configuration:

Backend Service:

- Build command: `pip install -r requirements.txt`
- Start command: `gunicorn app:app`
- Environment variables: `DATABASE_URL`, `FOOTBALL_DATA_API_KEY`

Frontend Static Site:

- Build command: `cd frontend && npm install && npm run build`
- Publish directory: `frontend/build`

PostgreSQL Database:

- Managed service on Render's infrastructure
- Automated daily backups
- Connection string provided as `DATABASE_URL` environment variable

Continuous deployment triggers on every push to GitHub `main` branch. Render detects changes, rebuilds the frontend and backend, and deploys new versions automatically. This automation reduces deployment friction—fixing bugs or adding features requires only committing code, not manual server management.

4.8 Summary

This chapter presented the technical design of football prediction system.The three-tier architecture separates frontend presentation,backend logic, and database storage,enabling independent development and deployment.Five RESTful API endpoints provide access to predictions ,historical results,model statistics,accuracy metrics, and live league data.The PostgreSQL database stores predictions in a single table optimized for common queries.Feature engineering transforms raw match data into 16 numerical features through historical aggregation and real-time API integration.The React frontend consists of four components (Dashboard, History, Models, Team Stats) that fetch data from the backend and render responsive UIs.Deployment on Render.com leverage platform-as-a-service automation for continuous deployment from GitHub.The next chapter evaluates the system's real-world performance across seven gameweeks, analyzing prediction accuracy and identifying patterns in successes and failures.

Chapter 5

System Evaluation

This chapter evaluates the football prediction system against the objectives established in Chapter 1. Evaluation encompasses quantitative analysis (prediction accuracy, feature importance, model comparison) and qualitative assessment (system reliability, deployment success, code quality). Real-world performance data from gameweeks 27-31 provides empirical evidence of strengths and limitations.

5.1 Prediction Accuracy Results

The primary success metric is prediction accuracy: the percentage of correct outcome predictions across all evaluated matches. Between February and April 2026, the system generated predictions for seven gameweeks spanning 68 matches. Overall accuracy was 36.76% (25 correct predictions out of 68 total).

Table 5.1 presents per-gameweek results.

Gameweek	**Correct**	**Total**	**Accuracy**
27	3	10	30.0%
28	4	10	40.0%
29	1	10	10.0%
30	3	10	30.0%
31	4	8	50.0%
32	4	10	40.0%
33	6	10	60.0%
Overall	**15**	**48**	**31.25%**

Table 5.1: Prediction accuracy by gameweek

Performance varied substantially, ranging from 10% (GW29) to 60% (GW33).This demonstrates both the challenge of football prediction and the limitations of eval-

uating small sample sizes—ten matches provide insufficient data to distinguish skill from random variation.Gameweek 33 achieved 60% accuracy, exceeding professional bookmaker benchmarks [1]. This suggests the model can perform competitively when match conditions align favorably with historical patterns. Four correct predictions out of eight suggest the model may have captured underlying patterns.

Conversely, Gameweek 29 dropped to 10% accuracy when five matches resulted in away wins versus only one predicted. Analysis revealed systematic home advantage overestimation: the model inherited historical bias from training data where home teams won approximately 45% of matches. Modern football exhibits reduced home advantage due to evolving tactics and professionalization, creating a distribution shift the model failed to account for.

The 36.76% overall accuracy falls short of the 40-50% target established in project objectives. However, this outcome provides valuable academic material. Unlike laboratory experiments where failure invalidates research, real-world deployment exposes models to conditions no amount of training data can fully capture: injuries, tactical surprises, referee errors, psychological factors. The gap between 59.74% training accuracy and 36.76% real-world performance quantifies the generalization challenge in sports prediction.

5.2 Comparison to Benchmarks

Contextualizing the 36.76% accuracy requires comparison to established benchmarks:

- **Naive baseline:** Predicting the most frequent outcome (home win) for every match yields approximately 45% accuracy on historical data. However, this strategy fails catastrophically when away teams dominate, as occurred in Gameweek 29.

- **Academic research:** Student-built models using public data typically achieve 40-50% accuracy [2, 3]. This project's 36.76% overall accuracy approaches this range, with Gameweek 33 achieving 60% — exceeding academic benchmarks.

- **Professional bookmakers:** Commercial systems with proprietary data (confirmed lineups, injury reports, betting market intelligence) reach 50-55% accuracy [1]. The 20-point gap (31% vs 51%) quantifies the value of insider information.

- **Best gameweek performance:** The 60% accuracy in Gameweek 33 exceeded bookmaker standards, demonstrating the model's capability under favorable conditions.

The results suggest that while the model underperformed academic averages overall, it demonstrated capability for competitive performance in individual gameweeks. Sustained accuracy at the 40-50% level would require addressing systematic biases (home advantage overestimation) and incorporating additional features (injury data, momentum indicators).

5.3 Feature Importance Analysis

Logistic Regression coefficients reveal which features contribute most to prediction accuracy. Table 5.2 ranks features by absolute coefficient magnitude.

Rank	Feature	Coefficient
1	HomePosition (form-based)	-2.23
2	AwayPosition (form-based)	1.49
3	AwayDrawRate	0.58
4	HomeTablePos (actual position)	0.37
5	HomeDrawRate	0.32
6	PositionGap (form-based)	-0.28
7	AwayForm5	0.24
8	GoalsCloseness	0.19
9	FormCloseness	-0.17
10	HomeForm5	0.15

Table 5.2: Top 10 features by Logistic Regression coefficient magnitude (remaining 6 features had coefficients < 0.15)

The most impactful feature is HomePosition (form-based ranking), with a strongly negative coefficient (-2.23). This counterintuitive result—where higher form-based position (worse ranking) decreases home win probability—likely reflects issues with how positional features are encoded. In league tables, position 1 is best and position 20 is worst. The negative coefficient suggests the model learned this inverse relationship: as home position number increases (team gets worse), home win probability decreases.

AwayPosition ranks second (+1.49), with positive coefficient indicating that as away position number increases (team gets worse), away win probability paradoxically increases. This appears to be another artifact of position encoding—the

model may be learning the inverse of intended relationships due to inconsistent feature scaling.

HomeTablePos (actual league position from API) ranks fourth despite being hypothesized as the most informative feature. Its modest coefficient (0.37) suggests either: (1) the API position provides limited additional signal beyond form-based proxies, or (2) the model weighted form-based position more heavily due to that feature's greater variance in training data.

Draw rate features (AwayDrawRate, HomeDrawRate) rank highly, validating the hypothesis that teams with high draw tendencies can be identified through historical patterns. However, as noted in Chapter 1, draw prediction remained challenging—the model predicted draws infrequently despite these features.

Goal-based features (GoalsCloseness, FormCloseness) ranked lower, suggesting that positional and form-based features capture most predictive signal. Average goals scored and conceded, while intuitively relevant, correlate strongly with league position—top teams score more and concede less—creating redundancy.

5.4 Model Comparison

Three algorithms were trained and evaluated on identical data: Logistic Regression, Random Forest (200 trees), and Decision Tree. Table 5.3 compares their training accuracy on the held-out test set.

Algorithm	**Test Accuracy**
Logistic Regression	59.74%
Random Forest (200 trees)	49.33%
Decision Tree	37.07%

Table 5.3: Model comparison on historical test data

Logistic Regression outperformed both ensemble and tree-based methods by approximately 10 percentage points. This outcome was initially surprising—ensemble methods like Random Forest typically outperform linear models on complex problems. The result suggests that for this problem, well-engineered linear features (form, position, goals) capture most predictive signal, and additional non-linear interactions provide limited benefit.

Random Forest's underperformance may stem from the relatively small training set (628 matches). Ensemble methods typically require large datasets to learn complex patterns without overfitting. With only 628 training examples split across three outcome classes (home/draw/away), each tree sees approximately 300-400 matches per bootstrap sample—potentially insufficient for robust pattern learning.

Decision Tree's poor performance (37.07%) reflects overfitting. Single trees, when allowed to grow deep, memorize training data by creating nodes matching individual examples. Pruning techniques (limiting depth, requiring minimum samples per leaf) would likely improve performance, but this was not explored given time constraints.

The 59.74% training accuracy achieved by Logistic Regression establishes an upper bound for this feature set and training data. Even the best model exceeded random guessing by a meaningful margin—a naive random classifier would achieve approximately 33% accuracy across three outcome classes (home win, draw, away win)—reinforcing the fundamental difficulty of football prediction.The 23-point gap between training accuracy (59.74%) and real-world accuracy (36.76%) quantifies the generalization challenge.

5.5 Case Study: Gameweek 29 Failure

Gameweek 29 produced the lowest accuracy (10%, 1 correct out of 10) and warrants detailed analysis. Table 5.4 compares predictions to actual results.

Match	**Predicted**	**Actual**	**Correct?**
Aston Villa vs Chelsea	Home Win	Away Win	×
Brighton vs Arsenal	Away Win	Away Win	✓
Fulham vs West Ham	Draw	Away Win	×
Man City vs Nott'm Forest	Home Win	Draw	×
Newcastle vs Man Utd	Draw	Home Win	×
Spurs vs Crystal Palace	Draw	Away Win	×
Bournemouth vs Brentford	Away Win	Draw	×
Everton vs Burnley	Away Win	Home Win	×
Wolves vs Liverpool	Away Win	Home Win	×
Leeds vs Sunderland	Home Win	Away Win	×

Table 5.4: Gameweek 29 predictions versus actual results

The pattern reveals the model's confusion across all outcome types. The model predicted 3 home wins, 4 draws, and 3 away wins—reasonably balanced distribution. However, actual results were 2 home wins, 2 draws, and 6 away wins. Only one prediction succeeded: Brighton vs Arsenal, where the model correctly predicted an away win for Arsenal (ranked 1st) over Brighton (ranked 7th).

The failures demonstrate systematic issues:

Draw over-prediction: Four matches were predicted as draws (Fulham vs West Ham, Man City vs Nott'm Forest, Newcastle vs Man Utd, Spurs vs Crystal Palace). Only Man City vs Forest actually ended in a draw—the other three

resulted in decisive outcomes (two away wins, one home win). The model's draw threshold (max probability $< 40\%$) triggered too frequently, predicting uncertainty where clearer outcomes occurred.

Home advantage misapplication: Several home teams predicted to win (Aston Villa, Man City, Leeds) either drew or lost. For example, Aston Villa (ranked 4th) was predicted to beat Chelsea (ranked 6th) at home, but Chelsea won away. The model weighted generic home advantage over the modest two-position gap.

Away win under-prediction: Six matches ended in away wins, yet the model predicted only three away wins (Brighton-Arsenal, Bournemouth-Brentford, Everton-Burnley, Wolves-Liverpool). Critical misses included Spurs vs Crystal Palace (predicted draw, Palace won away) and Leeds vs Sunderland (predicted home win, Sunderland won away).

This gameweek featured several upsets where lower-ranked teams defeated higher-ranked opponents (Crystal Palace over Spurs, Chelsea over Aston Villa), conditions the model handles poorly. When form-based features conflict with league positions or when underdogs perform above expectations, prediction accuracy collapses.

5.6 Case Study: Gameweek 31 Success

Gameweek 31 achieved 50% accuracy (4 correct out of 8), matching industry benchmarks. Analyzing successful predictions reveals conditions under which the model performs well.

Of the four correct predictions:

- **Newcastle vs Sunderland (Away Win predicted, Away Win actual):** Newcastle (ranked 3rd) heavily favored over Sunderland (ranked 11th). Large positional gap with Newcastle's superior form. Model correctly identified away team superiority overriding home advantage.

- **Everton vs Chelsea (Home Win predicted, Home Win actual):** Everton (ranked 8th) vs Chelsea (ranked 6th). Everton's stronger recent form tilted prediction toward home win despite similar league positions. Final score 3-0 to Everton.

- **Fulham vs Burnley (Home Win predicted, Home Win actual):** Fulham (9th, 44 points) vs Burnley (20th, 20 points). Massive quality gap—bottom-placed Burnley facing mid-table Fulham. Fulham won 3-1 as predicted.

- **Brighton vs Liverpool (Home Win predicted, Home Win actual):** Brighton (6th) hosting Liverpool (5th). Model predicted home win based on Brighton's home form advantage. Brighton upset Liverpool 2-1, demonstrating accurate assessment of home advantage strength.

The four incorrect predictions:

- **Spurs vs Nott'm Forest (Home Win predicted, Away Win actual):** Spurs (17th) vs Forest (16th). Model predicted home win based on position gap and home advantage, but Forest upset—demonstrating football's unpredictability.

- **Leeds vs Brentford (Away Win predicted, Draw actual):** Close prediction—model favored away team but match ended drawn. Brentford's away form (only 4 points from last 5 away) didn't justify confidence.

- **Aston Villa vs West Ham (Away Win predicted, Home Win actual):** Model predicted West Ham away win despite Villa's higher position (4th vs 18th). Form features likely dominated—West Ham may have had recent momentum that didn't materialize.

- **Bournemouth vs Man Utd (Home Win predicted, Draw actual):** Another near-miss. Model predicted Bournemouth home win, match ended 2-2 draw.

Common patterns in successful predictions:

1. **Large quality gaps:** When league position difference exceeded 7-8 places combined with point differentials (Fulham 44pts vs Burnley 20pts, Newcastle vs Sunderland), predictions succeeded.

2. **Form alignment:** When recent form (last 5 matches) aligned with league standing, confidence increased and predictions proved correct (Everton's form advantage over Chelsea).

3. **Clear favorites:** Matches with unambiguous favorites—whether home or away—produced correct predictions. Ambiguous matchups (similar positions, conflicting form) remained challenging.

The 50% accuracy demonstrates the model's ceiling: given favorable conditions (clear quality gaps, form alignment, decisive favorites), performance matches professional standards. Sustaining this accuracy across multiple gameweeks would require addressing systematic biases and incorporating additional features.

5.7 Assessment Against Objectives

Chapter 1 established six project objectives. This section evaluates success against each:

Objective 1: Build a machine learning model with 40-50% real-world accuracy

Status: Partially achieved. The model was built and deployed, achieving 36.76% overall accuracy, approaching the 40-50% target. Gameweek 33 reached 60%, exceeding industry-standard accuracy under favorable conditions.

Objective 2: Engineer meaningful features capturing team performance

Status: Achieved. Sixteen features spanning form metrics, goal statistics, positional data, and draw rates were successfully engineered. Feature importance analysis revealed that positional features contributed most to predictions, validating design decisions.

Objective 3: Integrate external data sources via API

Status: Achieved. The football-data.org API was successfully integrated for live league standings and match results. Real-time data fetching proved reliable across 68 predictions with minimal rate-limiting issues.

Objective 4: Deploy a production-ready web application

Status: Achieved. The system deployed on Render.com with continuous deployment from GitHub. The application remained accessible throughout the evaluation period with no downtime. Four frontend pages (Dashboard, History, Models, Team Stats) provided intuitive interfaces for viewing predictions and results.

Objective 5: Evaluate real-world performance across multiple gameweeks

Status: Achieved. Seven gameweeks (27-33) spanning 68 predictions provided sufficient data to identify patterns. Variability across gameweeks (10-60%) revealed both strengths and limitations, offering more insight than single-gameweek evaluation.

Objective 6: Identify model limitations through analysis

Status: Achieved. Systematic home advantage overestimation, draw prediction failure, and sensitivity to training data distribution were identified. Case studies of Gameweek 29 (failure) and Gameweek 31 (success) illuminated conditions affecting performance.

Four of six objectives were fully achieved, one was partially achieved (accuracy target), and all provided valuable learning outcomes. The shortfall in prediction accuracy, while disappointing from a performance perspective, generated insights into ML limitations that purely achieving the target would not have revealed.

5.8 Identified Limitations

Several limitations constrain the system's performance and scope:

Limited Feature Set: The model uses only aggregated team statistics (form, goals, positions) without player-level data. Injuries to key players, suspensions, and lineup changes significantly impact match outcomes but are invisible to the model. Professional systems incorporate confirmed starting lineups announced hours before kickoff—information unavailable when predictions are generated days in advance.

Training Data Recency Bias: Using only the last three seasons (2,261 matches) balances recency against sample size. However, this restricts the model's exposure to diverse tactical eras and team compositions. Older data may be less relevant due to changing playing styles, but excluding it limits pattern recognition.

Class Imbalance Handling: The threshold-based draw detection (predict draw if max probability $< 40\%$) is a simple heuristic. More sophisticated techniques—SMOTE oversampling, class weighting, or ensemble methods designed for imbalanced data—might improve draw prediction without extensive engineering.

Home Advantage Overestimation: The model inherited historical home advantage bias from training data.Modern football exhibits reduced home-field advantage due to professionalization, tactical evolution, and potentially lingering effects of COVID-era empty stadiums. Explicitly modeling time-varying home advantage or downweighting older training examples might address this.

Small Evaluation Sample: Sixty-eight predictions across seven gameweeks provide limited statistical power. Random variance can dramatically affect accuracy—a single gameweek of ten matches can swing overall results by several percentage points. Evaluating across an entire season (380 matches) would reduce noise.

No Momentum Features: The model uses fixed lookback windows (last 5 matches) without weighting recent matches more heavily. Teams on winning streaks or slumps may exhibit momentum not captured by simple point totals. Exponential decay weighting or streak-specific features might improve performance.

Despite these limitations, the system successfully demonstrated end-to-end machine learning deployment: data collection, feature engineering, model training, API integration, frontend development, and real-world evaluation. The identified limitations provide clear directions for future work.

5.9 Summary

This chapter evaluated the football prediction system against established objectives and benchmarks.Across 68 predictions spanning seven gameweeks, the sys-

tem achieved 36.76% overall accuracy, approaching the 40-50% academic target but providing valuable insights into ML limitations.Performance varied substantially by gameweek (10-60%), with Gameweek 31 matching professional benchmarks.Feature importance analysis revealed that positional features contributed most to predictions,validating API integration decisions. Model comparison showed Logistic Regression (59.74% training accuracy) outperforming Random Forest and Decision Tree, suggesting well-engineered linear features capture much of the predictive signal. Case studies of Gameweek 29 (10% accuracy) and Gameweek 31 (50% accuracy) illuminated conditions affecting performance: the model succeeds on mismatches with large positional gaps but struggles when features conflict or home advantage bias dominates. Four of six project objectives were fully achieved (feature engineering, API integration, deployment, evaluation); prediction accuracy was partially achieved. Identified limitations—limited features, class imbalance, home advantage overestimation, small evaluation sample—provide clear directions for future work. The next chapter concludes the dissertation by reflecting on lessons learned and proposing improvements.

Chapter 6

Conclusion

This dissertation presented the design,implementation,and evaluation of an AI-powered Premier League match prediction system.The project demonstrated end-to-end machine learning deployment: data collection from multiple sources,feature engineering transforming raw statistics into predictive signals,model training and comparison,RESTful API development,frontend interface design,and production deployment with continuous integration.Real-world evaluation across seven game-weeks (68 predictions) provided evidence of both capabilities and limitations,generating insights that purely optimizing training accuracy would not have revealed.

6.1 Project Summary

The system was built iteratively over three months during the 2025-26 academic year. Development began with research into existing football prediction techniques and identification of publicly available data sources.A command-line prototype validated the core concept:historical Premier League match data combined with live league standings could train machine learning models to predict outcomes.Subsequent iterations added backend infrastructure (Flask API,PostgreSQL database),frontend interfaces (React application with four pages), and deployment automation (GitHub integration with Render.com).The technical stack reflects pragmatic choices balancing ease of development, availability of libraries,and industry relevance.Python and scikit-learn provided battle-tested machine learning implementations.Flask enabled rapid API development without the overhead of enterprise frameworks. React demonstrated modern frontend skills employers value.PostgreSql offered relational data integrity with managed hosting eliminating database administration burdens. The entire system deploys on Render.com's platform-as-a-service,showcasing production-ready cloud deployment patterns.

Feature engineering proved more impactful than algorithm selection. Sixteen

features were engineered spanning team form (points from last 5 matches), goal statistics (average scored and conceded), positional data (league standings and form-based rankings), and draw tendencies (recent draw rates). Critically, positional features overall proved most predictive, with form-based rankings contributing the strongest signal and actual league positions from the API providing complementary value, validating the decision to integrate external data sources rather than relying solely on calculated aggregates.

Three machine learning algorithms were trained and compared: Logistic Regression, Random Forest (200 trees), and Decision Tree. Logistic Regression achieved the highest test accuracy (59.74%), outperforming Random Forest (49.33%) and Decision Tree (37.07%) by approximately 10 percentage points. This outcome—linear models outperforming ensembles—was initially surprising but suggests that well-engineered features capture most predictive signal available in the data. Complex non-linear interactions provide limited additional benefit when fundamental features (form, position, goals) already encode team quality comprehensively.

Real-world deployment began with Gameweek 27 in February 2026. The system generated predictions for upcoming matches, stored them in PostgreSQL with probability distributions, and compared predictions against actual results published after each gameweek concluded. Across seven gameweeks spanning 68 predictions, overall accuracy was 36.76% (25 correct predictions). This approaches the 40-50% academic benchmark, with Gameweek 33 achieving 60%—exceeding professional bookmaker standards—and provides valuable insights into machine learning limitations when deployed against genuinely unpredictable phenomena.

6.2 Key Findings

Several significant findings emerged from system evaluation:

6.2.1 Prediction Accuracy Exhibits High Variance

Accuracy varied dramatically across gameweeks: 30% (GW27), 40% (GW28), 10% (GW29), 30% (GW30), 50% (GW31), 40% (GW32), and 60% (GW33).This volatility—ranging from complete failure to industry-standard performance—demonstrates the challenge of football prediction and the danger of evaluating models on small samples. A single atypical gameweek (GW29, where 5 away wins occurred versus 1 predicted) can dramatically skew results. Sustained evaluation across an entire season (380 matches) would reduce noise, but even then, football's inherent unpredictability ensures accuracy will fluctuate.

Gameweek 33's 60% accuracy exceeded professional bookmaker benchmarks [1], proving the model can perform competitively under favorable conditions. The model performs best when multiple features align—position, form, and goals all favoring one outcome.

6.2.2 Positional Features Dominate Importance

Feature importance analysis revealed that league position—both form-based rankings and actual standings from the API—contributed most to predictions. The top three features by Logistic Regression coefficient magnitude were HomePosition (form-based, -2.23), AwayPosition (form-based, +1.49), and AwayDrawRate (+0.58). Actual league position (HomeTablePos, +0.37) ranked fourth despite being hypothesized as most informative.

This finding has practical implications: investing development effort into real-time API integration proved worthwhile,but the marginal improvement over calculated proxies was smaller than expected.Form-based position rankings-computed entirely from historical match data without external API calls-provide substantial predictive signal.Future iterations might rely more heavily on calculated features to reduce API dependency and rate-limiting constraints.Goal-based features (average goals scored and conceded, goal difference) ranked lower in importance, likely due to correlation with league position. Top teams score more and concede less almost by definition, creating feature redundancy. More sophisticated goal metrics—expected goals (xG), shot conversion rates, or defensive efficiency—might provide independent signal, but require data sources beyond what public APIs offer.

6.2.3 Home Advantage Overestimation

The model systematically over-predicted home wins, particularly evident in Gameweek 29 where 7 home wins were predicted but only 2 occurred. Analysis revealed the model inherited historical home advantage bias from training data (approximately 45% home wins, 30% away wins, 25% draws). Modern football exhibits reduced home-field advantage due to tactical evolution, professionalization reducing travel fatigue impact, and potentially lingering effects of COVID-era empty stadiums normalizing away performances.

This distribution shift—training data reflecting one era's patterns, deployment data exhibiting different patterns—is a classic machine learning challenge. The model learned that "home teams usually win" from historical data but failed to adapt when this tendency weakened. Explicitly modeling time-varying home advantage or downweighting older training examples might address this, though im-

plementation would require careful feature engineering to avoid introducing artifacts.

6.2.4 Draw Prediction Remains Challenging

The model initially never predicted draws,defaulting exclusively to home or away wins due to class imbalance (draws comprise only 25% of training data). Implementing a threshold rule—predict draw if maximum probability $< 40\%$—improved draw detection but remained imperfect. Gameweek 29 predicted 4 draws; only 1 proved correct.The threshold approach is a crude heuristic that treats draw prediction as uncertainty ("I'm not confident, so predict draw") rather than genuine pattern recognition ("this match has draw-specific characteristics").

More sophisticated approaches-SMOTE oversampling to balance training classes,class weighting to penalize draw misclassifications more heavily, or dedicated draw classifiers trained specifically on draw-predictive features—might improve performance.However,draws may be fundamentally harder to predict than wins.Decisive outcomes(wins) result from sustained team quality difference,while draws often stem from tactical cancellations,defensive stalemates, or late equalizers—factors difficult to model from aggregated statistics alone.

6.3 Project Outcomes

This project delivered the following concrete outcomes:

- **Deployed web application:** Publicly accessible at `https://fyp-frontend-wgcl.onrender.com`, demonstrating production-ready deployment with continuous integration from GitHub
- **Machine learning pipeline:** Trained Logistic Regression model achieving 59.74% accuracy on historical test data, with comparison to Random Forest and Decision Tree alternatives
- **Feature engineering framework:** Sixteen features calculated from historical match data and live API integration, with importance analysis revealing position-based features dominate
- **RESTful API backend:** Five Flask endpoints providing predictions, historical results, model statistics, accuracy tracking, and live league standings
- **React frontend:** Four pages (Dashboard, History, Models, Team Stats) offering intuitive interfaces for viewing predictions and analyzing performance

- **Real-world evaluation:** Sixty-eight predictions across seven gameweeks with 36.76% overall accuracy and a best gameweek of 60%, identifying systematic biases and performance patterns
- **Database schema:** PostgreSQL table storing predictions with probability distributions, actual results, and correctness flags enabling longitudinal analysis
- **Comprehensive documentation:** GitHub repository with README, code comments, and this dissertation explaining design decisions and evaluation results

Beyond technical deliverables, the project achieved learning outcomes spanning machine learning (supervised classification, feature engineering, model comparison), software engineering (API design, database modeling, frontend development), and deployment (cloud hosting, continuous integration, environment management). These skills directly transfer to industry positions in data science, software development, and machine learning engineering.

6.4 Unexpected Discoveries

Several discoveries emerged during development and evaluation that were not anticipated at project outset:

Linear models outperformed ensembles: The expectation was that Random Forest with its capacity for non -linear interactions and feature combinations,would outperform simple Logistic Regression.The opposite occurred—Logistic Regression achieved 59.74% versus Random Forest's 49.33%. This suggests that for football prediction with team-level aggregates, relationships are predominantly linear. Form, position, and goals combine additively rather than multiplicatively. Alternatively, the 628-match training set may be too small for Random Forest to effectively learn complex patterns without overfitting.

API integration provided modest improvement: Considerable development effort went into integrating the football-data.org API for real-time league standings.Feature importance analysis revealed actual positions (from API) ranked fourth,behind form-based position proxies (calculated from historical data).The marginal improvements-while statistically significant—was smaller than expected.This discovery suggest future iterations could reduce API dependency, relying more on calculated features to avoid rate limiting and simplify deployment.

Single gameweeks provide minimal signal: With only 8-10 matches per gameweek, random variance dominates. Gameweek 29's 10% accuracy and Gameweek 33's 60% accuracy represent extremes of small-sample noise.Even if the un-

derlying model has 40% true accuracy,individual gameweeks will vary from 0-70% purely by chance. This emphasizes the importance of longitudinal evaluation—conclusions drawn from single gameweeks are unreliable.

Deployment reveals issues invisible in development: Testing on historical data during development gave no indication of home advantage overestimation or draw prediction failure.These issues only became apparent when deploying against live matches.Real-world deployment exposes models to distribution shifts,edge cases,and operational challenges (API rate limits,cold starts, database connection failures) that laboratory testing cannot simulate.The value of empirical validation-actually running the system in production-cannot be overstated.

Low accuracy generates valuable insights: The 36.76% overall accuracy, while below the academic target, generated richer learning than purely achieving the 40-50% target might have.Analyzing failures (Gameweek 29's home advantage bias,draw over-prediction) provided insights into model limitations,feature engineering gaps,and fundamental unpredictability of football.Success often obscures weaknesses; failure forces confrontation with limitations.

6.5 Future Work

Several avenues for future investigation emerged from this project:

Momentum and streak features: The current model uses fixed 5-match lookback windows without weighting recent matches more heavily. Teams on winning or losing streaks may exhibit momentum not captured by simple point totals. Exponential decay weighting (recent matches weighted 2x, older matches 1x) or explicit streak detection (consecutive wins/losses) might improve predictions for teams in exceptional form.

Injury and suspension data: Professional prediction systems incorporate confirmed lineups announced hours before kickoff.While this information is unavailable days in advance when predictions are generated,historical injury impact could be modeled.Features encoding "number of key players injured" or "defensive injuries vs offensive injuries" might provide signal if injury can be sourced from football APIs.

Expected goals (xG) integration: Expected goals-a metric estimating shot quality based on position, angle, and game state—provides deeper insight than raw goal counts. A team scoring 2 goals from 1.0 xG is lucky; a team scoring 1 goal from 2.5 xG is unlucky. Incorporating xG data (if available via API) might improve predictions by capturing underlying performance quality rather than result-dependent statistics.

Ensemble methods combining models: Rather than selecting a single best algorithm,ensemble approaches combine multiple models' predictions.Logistic

Regressions excels on clear-cut matches; perhaps Random Forest performs better on ambiguous fixtures.A meta-classifier learning when to trust each model might achieve higher overall accuracy than any individual algorithm.

Time-varying home advantage: Explicitly modeling how home advantage has changed over time-rather than assuming a constant effect-could address the observed overestimation.A feature encoding "era" (pre-2020, COVID-era, post-COVID) or a decay function reducing historical home advantage influence might improve calibration.

Transfer learning from other leagues: Training data is limited to Premier League matches ,but tactical patterns,team dynamics, and statistical relationships exist across all football leagues.Transfer learning —pre-training on matches from Serie A, La Liga, Bundesliga, then fine-tuning on Premier League data—might improve generalization by exposing the model to broader tactical diversity.

Confidence-based filtering: Rather than displaying all predictions, showing only high-confidence predictions (e.g., probability > 60%) might improve perceived accuracy. Analysis in Chapter 5 suggested accuracy improves when filtering low-confidence predictions, trading coverage for precision. A user-facing "confidence threshold" slider could let users choose their own precision-coverage tradeoff.

6.6 Reflection

The project demonstrated that machine learning can provide value even accuracy falls short of targets.The 36.76% real-world accuracy approaches academic benchmarks (40-50%), with individual gameweeks reaching 60% — exceeding professional systems (50-55%),yet the project succeeded in its broader objective:building a complete,deployed system that generates prediction,tracks accuracy,and provides insights into both model capabilities and football's fundamental unpredictability. The gap between 59.74% training accuracy and 36.76% real-world accuracy quantifies the generalization challenge that defines applied machine learning.Models learns patterns from historical data,but the world changes:tactics evolve,team compositions shift ,individual player form fluctuates,injuries occur.No amount of historical data can predict genuinely novel events-refereeing controversies,managerial sackings mid-season,weather disruptions.Acknowledging these limitations is not failure:it is honest assessment of AI's appropriate role.Football prediction serves as a microcosm for machine learnings broader challenges: noisy data,class imbalance,distribution shift,small sample sizes, and irreducible uncertainty.The lessons learned-feature engineering matters more than algorithm selection,real-world deployment reveals issues invisible during development,low accuracy can generate valuable insights-transfer to other domains.Whether predicting customer churn (customers leaving a service), diagnosing medical conditions, or forecasting market

trends, practitioners face analogous challenges. The project also highlighted the value of full-stack competencies.Building a complete system-data collection,feature engineering,model training,API development,frontend design,database management,deployment automation-required integrating knowledge across multiple domains.Modern software engineering increasingly demands this breadth:specialists who can also communicate across disciplines. The ability to train a model (data science), expose it via API (backend engineering), display results in a responsive UI (frontend development), and deploy on cloud infrastructure (DevOps) distinguishes industry-ready graduates from those with purely theoretical knowledge.From a personal perspective,the project reinforced that setbacks often provide richer learning than successes.Gameweek 29's 10% accuracy forced deep analysis of home advantage bias and feature weighting issues that achieving 40% might have hidden.The draw prediction failure prompted investigation of class imbalance techniques.The gap between Logistic Regression and Random Forest performance challenged assumptions about ensemble superiority.These struggles-more than smooth successes-drive understanding.

6.7 Final Remarks

This dissertation presented an end-to-end machine learning system for Premier League match prediction. The system achieved 36.76% accuracy across 68 real-world predictions, approaching academic benchmarks and exceeding them in individual gameweeks but demonstrating both technical competence and critical awareness of limitations. Feature engineering, real-time API integration, production deployment, and empirical evaluation showcased skills directly applicable to industry roles. Identified limitations—home advantage overestimation, draw prediction challenges, small evaluation samples—provide clear directions for future work.Beyond metrics and deliverables,the project demonstrated that artificial intelligence serves best as a tool for insight rather than certainty.Football's unpredictability —where relegation-threatened teams upset title contenders, where tactical adjustments swing matches, where individual brilliance or errors decide outcomes—resists purely data-driven prediction. Machine learning can identify patterns, quantify tendencies, and estimate probabilities, but cannot eliminate uncertainty inherent in complex, competitive systems involving human agency. The value of this project lies not in achieving 50% accuracy but in building a complete system, deploying it against real-world data, analyzing failures honestly, and extracting insights applicable beyond football prediction. These outcomes—technical skills, critical thinking, empirical validation, and humility about AI's limitations—represent success regardless of accuracy percentages. Future students undertaking similar projects should prioritize deployment and real-world

testing over purely optimizing training metrics. Build systems that actually work in production, expose them to genuine use cases, and learn from failures. The gap between laboratory success and real-world performance is where the most valuable learning occurs.

Bibliography

[1] Anthony C Constantinou and Norman E Fenton. Solving the problem of inadequate scoring rules for assessing probabilistic football forecast models. *Journal of Quantitative Analysis in Sports*, 8(1), 2012.

[2] Rohit Baboota and Harleen Kaur. Predictive analysis and modelling football results using machine learning approach for english premier league. *International Journal of Forecasting*, 35(2):741–755, 2019.

[3] Niek Tax and Yme Joustra. Predicting the dutch football competition using public data: A machine learning approach. In *Transactions on Knowledge and Data Engineering*, 2015.

[4] David W Hosmer Jr and Stanley Lemeshow. Applied logistic regression. *New York*, 1989.

[5] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.

[6] Anthony Joseph, Norman E Fenton, and Martin Neil. Predicting football results using bayesian nets and other machine learning techniques. *Knowledge-Based Systems*, 19(7):544–553, 2006.

[7] Rory P Bunker and Fadi Thabtah. A machine learning framework for sport result prediction. *Applied computing and informatics*, 15(1):27–33, 2019.

[8] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of machine learning research*, 12:2825–2830, 2011.

[9] Alice Zheng and Amanda Casari. *Feature engineering for machine learning: principles and techniques for data scientists*. O'Reilly Media, 2018.

[10] Miguel Grinberg. *Flask web development: developing web applications with python*. O'Reilly Media, 2018.

[11] Alex Banks and Eve Porcello. *Learning React: functional web development with React and Redux*. O'Reilly Media, 2018.

[12] Regina Obe and Leo Hsu. *PostgreSQL: up and running: a practical guide to the advanced open source database*. O'Reilly Media, 2017.

[13] Nathalie Japkowicz and Shaju Stephen. The class imbalance problem: A systematic study. *Intelligent data analysis*, 6(5):429–449, 2002.

[14] Richard Pollard. Home advantage in football: A current review of an unsolved puzzle. *The Open Sports Sciences Journal*, 1(1), 2008.

[15] Kerry S Courneya and Albert V Carron. The home advantage in sport competitions: A literature review. *Journal of sport and exercise psychology*, 14(1):13–27, 1992.

[16] Guido Van Rossum and Fred L Drake Jr. Python reference manual. *Centrum voor Wiskunde en Informatica Amsterdam*, 1995.

[17] Wes McKinney. *Python for data analysis: Data wrangling with Pandas, NumPy, and IPython*. O'Reilly Media, 2012.

[18] Roy T Fielding. Architectural styles and the design of network-based software architectures. *Doctoral dissertation, University of California, Irvine*, 2000.

[19] Michael Stonebraker and Lawrence A Rowe. The design of postgres. *ACM Sigmod Record*, 15(2):340–355, 1986.

Appendix A

Project Repository and Installation

A.1 Source Code Repository

The complete source code for this project is publicly available on GitHub:
`https://github.com/mohelmahi02/FYP`
The repository contains the following main components:

- `app.py` — Flask backend API with all five endpoints
- `train_model.py` — Model training script
- `make_predictions.py` — Prediction generation script
- `services/` — Backend services (database, feature engineering, results comparison)
- `models/` — Trained model `.pkl` files
- `data/` — Historical Premier League match data CSV
- `frontend/` — React application source code

A.2 Live Application

The deployed application is publicly accessible at:
`https://fyp-frontend-wgcl.onrender.com`
A screencast demonstration of the system is available at:
`https://go.screenpal.com/watch/cOfOqKnOks3`

A.3 Installation Instructions

A.3.1 Prerequisites

- Python 3.12+
- Node.js 18+
- PostgreSQL 16+

A.3.2 Backend Setup

```
git clone https://github.com/mohelmahi02/FYP.git
cd FYP
pip install -r requirements.txt
export DATABASE_URL="your_postgresql_connection_string"
python train_model.py
python app.py
```

A.3.3 Frontend Setup

```
cd frontend
npm install
npm start
```

A.3.4 Generating Predictions

```
python make_predictions.py
python update_csv_with_current_season.py
python -m services.compare_results
python -m services.evaluation_service
```